

Auto-Adaptive License Plate Recognition (ALPR) System

Technical Project Report

Prepared by: * Udish Arora (2023AIB1316) * Ekam Sethi (2023AIB1004)

1. Executive Summary

This project presents a fully autonomous, production-grade Automatic License Plate Recognition (ALPR) system built for real-world Indian traffic conditions. Unlike classical ALPR pipelines that assume clean, well-lit imagery, this system employs a **environmental classifier** to intelligently route each image through one of three specialised restoration engines before detection and recognition occur. The result is a pipeline that adapts itself to fog, darkness, and blur without requiring human intervention or separate classifier model training.

The system is packaged as a containerised microservices application built on docker, with a FastAPI gateway, asynchronous Celery workers, a React/TypeScript frontend, and a full observability stack (Prometheus + Grafana). It incorporates active learning for continuous model improvement and exposes comprehensive MLOps telemetry.

2. Problem Statement

Standard ALPR systems fail under adverse conditions:

Condition	Failure Mode
Low light / Night	Characters wash out; contrast too low for detection
Fog / Haze	Atmospheric scattering blurs edges; DNN confidence drops
Motion Blur	Deblurring required before any text recognition is possible

Indian roads amplify these challenges through highly variable ambient lighting, monsoon seasons, and mixed traffic speeds. This project addresses all four failure modes through an adaptive, deterministic routing strategy.

3. System Architecture

The system is organised as seven Docker services communicating over a shared Redis message queue:



Asynchronous Design

The critical design decision is the **async fire-and-poll pattern**:

1. `POST /process-image` → saves file, dispatches Celery task, returns `task_id` immediately
2. Frontend polls `GET /status/{task_id}` until `status = "SUCCESS"`
3. Results (base64 images + metadata) are stored in Redis by Celery and fetched on success

This prevents HTTP timeouts on long-running ML inference (NAFNet and TrOCR can take 5–20 seconds on CPU).

4. Pipeline Stages

Stage 1 — Environmental Classification (classifier.py / image_routing.py)

The classifier is a **zero-parameter, deterministic routing engine** built entirely from OpenCV statistics. No separate ML model is required. Three signals are computed:

Signal 1: Average Pixel Brightness

```
brightness = mean(grayscale_image)
```

If `brightness < 60` → route to DarkIR (low-light restoration).

Signal 2: Dark Channel Prior (DCP)

```
dark_channel = erode(min(R, G, B channels), 15×15 kernel)
avg_dcp = mean(dark_channel)
```

If `avg_dcp > 110` and image is generally bright → route to DeHaze. The DCP exploits the physical property that haze adds uniform atmospheric light across all channels, raising the minimum channel value.

Stage 2 — Image Restoration

2a. NAFNet (Nonlinear Activation Free Network)

- **Purpose:** General motion blur removal and super-resolution
- **Library:** `nafnetlib` (wraps pre-trained NAFNet weights)
- **Model:** `reds_width64` pre-trained on REDS video deblurring benchmark
- **Lazy loading:** `@lru_cache` ensures the model is loaded into memory only once and reused across all requests processed by the same Celery worker

2b. DarkIR (Low-Light Restoration)

- **Purpose:** Restore structural detail from deeply underexposed night-time images

- **Architecture:** Custom `DarkIR` convolutional network trained on the LOLBlur dataset (Low-Light with Blur)

2c. DeHaze (Dark Channel Prior Method)

- **Purpose:** Remove atmospheric haze and fog
 - **Algorithm:** Fully mathematical implementation of the He et al. Dark Channel Prior: 1. Compute dark channel `J_dark` 2. Estimate atmospheric light `A` from the brightest pixels in the dark channel 3. Estimate transmission map `t(x)` 4. Refine with a guided filter (edge-preserving smoothing) 5. Recover scene radiance: $J(x) = (I(x) - A) / t(x) + A$
 - **No neural network required** — entirely OpenCV + NumPy
-

Stage 3 — License Plate Detection (`detector.py`)

- **Model:** YOLOv8 (`yolov8_plate.pt`) — fine-tuned specifically for Indian license plates
 - **Confidence threshold:** 0.25 (permissive to maximise recall)
 - **Input handling:** Automatically upgrades grayscale images to 3-channel RGB before inference
 - **Output:** List of bounding boxes with `{box: [x1,y1,x2,y2], score: float, class_id: int}`
 - **Selection:** The bounding box with the highest confidence score is chosen as the best plate
-

Stage 5 — Optical Character Recognition (`ocr_engine.py`)

- **Model:** Microsoft TrOCR (`trocr-base-printed`) — a Vision Encoder-Decoder Transformer
 - **Encoder:** ViT (Vision Transformer) — extracts visual features from the plate image
 - **Decoder:** RoBERTa — autoregressively generates character tokens from features
 - **LoRA Adapter Integration:** On startup, the system checks for LoRA adapter weights. If found, `PeftModel` wraps the base model and `merge_and_unload()` fuses the adapter weights for zero-overhead inference. This enables continuous model improvement via fine-tuning without redeploying the base model.
 - **Indian Plate Conditioning:** A strict post-processing step enforces the Indian plate format (`SS NN LL NNNN` — 2 state letters, 2 RTO digits, 2 series letters, 4 registration digits), correcting common OCR confusions (O↔0, I↔1, S↔5, etc.)
 - **Confidence Scoring:** Average softmax probability across all generated tokens
-

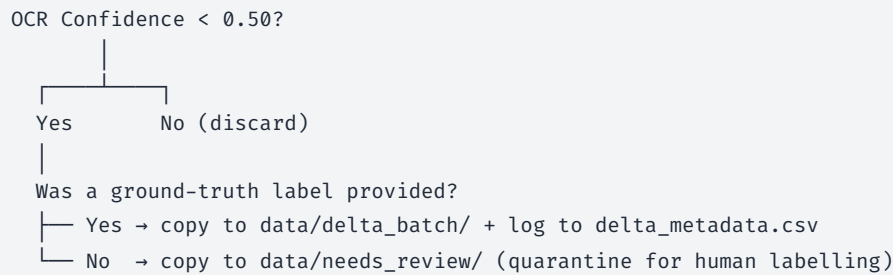
Stage 6 — RTO Metadata Parsing (`rto_metadata.py`)

Given the recognised plate string (e.g., `MH12AB1234`): - First 2 characters → state abbreviation → lookup against all 36 Indian state/UT codes - Characters 3-4 → RTO district code (numeric) -

Output: `{state: "Maharashtra", district_code: "12"}`

5. Active Learning & Continuous Training

The system implements an **active learning loop** that automatically handles low-confidence predictions:



The `delta_batch` constitutes a continuously growing dataset of hard examples (genuine ambiguous real-world plates). Once the `delta_batch` exceeds 100 images, the system automatically triggers a LoRA fine-tuning cycle. After the fine-tuning is complete, the new LoRA adapter weights are hot-swapped and merged into the base model dynamically. This redeployment happens seamlessly in the background with zero downtime, ensuring the application remains fully available without affecting active traffic while the model continuously improves itself.

6. Data Versioning

The system utilises **Data Version Control (DVC)** for versioning machine learning models and datasets alongside the codebase. A DVC remote is configured pointing to an **Amazon S3 bucket**, ensuring that large artifacts (such as model weights, LoRA adapters, and augmented image datasets) are securely backed up, tracked across iterations, and efficiently pulled during the CI/CD deployment process.

7. MLOps Instrumentation

Prometheus metrics are exported at `/metrics`. **Grafana** then automatically accesses these metrics via the Prometheus datasource to create real-time, interactive monitoring dashboards. Key metrics include:

Metric	Type	Purpose
<code>alpr_ocr_confidence</code>	Gauge	Live OCR confidence; sudden drops signal data drift
<code>alpr_data_drift_score</code>	Gauge	Compound proxy: $100/\text{blur} + \text{brightness}-128 + (1-\text{conf}) \times 100$
<code>alpr_inference_latency_seconds</code>	Histogram	End-to-end pipeline timing
<code>alpr_delta_queue_size</code>	Gauge	Continuous Training (CT) queue depth

8. CI/CD Pipeline

The project implements a robust automated CI/CD pipeline orchestrated via **GitHub Actions**. To handle the heavy compute requirements of machine learning model builds and deployments without incurring excessive cloud limits, the pipeline relies exclusively on **self-hosted runners**. These runners automatically build the Docker images, run sanity checks, and handle the continuous deployment process whenever new code or active-learning models are merged.

9. Deployment

The entire stack is defined in `docker-compose.yml` and starts with a single command:

```
cd ALPR-app
docker-compose up --build
```

Service URL	Purpose
<code>http://localhost:8080</code>	Main application (via Nginx proxy)
<code>http://localhost:8000/docs</code>	FastAPI Swagger UI
<code>http://localhost:9090</code>	Prometheus metric explorer
<code>http://localhost:3000</code>	Grafana dashboard

Persistent Volumes: - `hf_cache` — HuggingFace model weights (avoids re-download on restart) - `lora_weights` — LoRA adapter checkpoints (survives container deletion) - `prometheus_data` / `grafana_data` — Metric time-series and dashboards

10. Limitations & Future Work

Limitation	Proposed Solution
YOLO may miss severely tilted plates	Rotate-and-retry detection at $\pm 15^\circ$, $\pm 30^\circ$
TrOCR runs on CPU only	Add <code>device="cuda"</code> path; profile on GPU
Single-class YOLO (plates only)	Add vehicle bounding box detection for multi-vehicle scenes

11. Technology Stack

Layer	Technology
API Gateway	FastAPI (async ASGI)
Task Queue	Celery 5
Message Broker / Result Backend	Redis
Object Detection	YOLOv8 (Ultralytics)
Image Deblurring	NAFNet (nafnetlib)
Low-Light Restoration	DarkIR (custom PyTorch architecture)
Fog Removal	Dark Channel Prior (OpenCV / NumPy)
OCR	Microsoft TrOCR (HuggingFace Transformers)
Fine-Tuning	LoRA (PEFT library)
Image Processing	OpenCV, Pillow, NumPy
Frontend	React + TypeScript (Vite)
Reverse Proxy	Nginx
Monitoring	Prometheus + Grafana
Containerisation	Docker + Docker Compose
Data Versioning	DVC + Amazon S3
CI/CD Pipeline	GitHub Actions (Self-Hosted Runners)